Checking your data: quality control in pyflic

A guide for researchers analysing FLIC recordings with pyflic. It explains every quality-control (QC) check pyflic runs, the files and figures each one writes, how to read them, and how to decide which chambers to leave out.

Sections 1 to 5 apply to every experiment. Section 6 adds the light QC of optogenetic experiments, and section 7 the checks a DFM firmware 4.0.0 recording adds to it: each Paradigm checked against its own rule. Section 8 covers the checks of a Progressive Ratio experiment, and section 9 what a Project does with its members' QC. Section 10 is a checklist to work through for each recording, and section 11 a troubleshooting table.

You do not need to read code to use it. For installing and starting pyflic, see Getting Started; if you installed it that way, type the `pyflic …` commands below as `uv run pyflic …`, from the pyflic folder. For the Progressive Ratio assay itself, see Analysing a Progressive Ratio assay. The in-app help (`pyflic help`, or **F1** in any window) covers the same ground as a concise reference. Its reading path *Checking your data* starts from the QC Viewer.

---

1. What QC is for

Every number pyflic reports rests on two things: the signal each well recorded, and the licks pyflic detected in it. Both can go wrong in ways that still produce plausible results. A well whose background drifts, thresholds that miss brief licks, two wells that pick up each other's signal: none of these stops an analysis, and each one changes its numbers.

QC asks six questions:

Question	Where to look	Section
Did the recording run cleanly?	the integrity report and the data breaks	3.1, 3.2
Does detection agree with what you would call by eye?	the signal plots and the QC Viewer	3.3, 4
Are the two wells of a chamber independent?	simultaneous feeding and bleeding (two-well)	3.4, 3.5
Is each chamber a usable fly?	the exclusion cutoffs and your own inspection	5
Was the light where the licks were?	the optogenetic light QC	6
Did the light follow its program's rule?	the rule checks (firmware 4.0.0)	7

Three rules run through everything:

- **Do QC before statistics.** Once you have seen a result, every exclusion looks like a choice made for it.
- **Most checks flag; you decide.** Only the design's exclusion cutoffs remove chambers on their own. Everything else is information for your judgement.

- **Apply one rule to every chamber, whatever its treatment.** A rule applied to one treatment only biases the comparison.

---

2. Running QC

The QC panel

In the Hub, load the member (double-click its row in the Project panel, or open a standalone experiment with `pyflic hub "path/to/my_experiment"`), then click the **QC** tile. Its panel has two cards.

The QC card lists everything **Run QC** can write, one checkbox per item, in groups:

- **QC reports**, for every member: *Data integrity*, *Data breaks*, *Raw signal plots*, *Baselined signal plots* and *Cumulative licks plots*, and on a two-well layout *Simultaneous feeding* and *Bleeding between wells* (section 3);
- **Optogenetics**, when the member is optogenetic: *Opto light QC tables*, *Light explained by licks* and *Program timeline* (sections 6 and 7);
- **Progressive Ratio**, when the member's Experiment Type includes the Progressive Ratio module: *Light QC table*, *Licks per light event* and *Sucrose Well resting level* (section 8).

Run QC computes the QC once and writes each ticked item: the first two groups into `qc/`, the Progressive Ratio group into `analysis/`. Every box starts ticked, and with every box ticked Run QC writes the whole **QC bundle**: the integrity report, the data breaks, the three signal plots, the two-well crosstalk tables and, on an optogenetic member, the light QC and the Program timeline of each DFM that has one. A Progressive Ratio member's own light QC comes on top.

- **A tick is saved at once**, so every later run leaves out the same items. A standalone experiment keeps it in its `flic_config.yaml`, under `global.omit.qc`. A member of a Project keeps it in the Project's `project.yaml`, under `design.global.omit.qc`, so it applies to **every member**: members are checked alike. A script's `run_qc` step reads the same list.
- **A missing box** is an item the member cannot have, such as a two-well check on a single-well layout. One line under the groups names what is not offered; hover over it for the reasons.
- **A greyed box** is an item the data cannot answer: *Program timeline* when no DFM has a `Program.txt` section or firmware 4.0.0 data. Its tooltip gives the reason.
- **The log states what a run left out**, and, apart from that, what did not apply.
- **No run removes a file.** An item you untick keeps whatever an earlier run wrote, so a file in `qc/` can be older than the rest. Tick it and run again before you rely on it.

The View card runs nothing. It says when no QC has been written yet, and holds three buttons:

- **Open QC Viewer** opens the QC Viewer on the member the Hub has loaded (section 4).
- **View QC plots** opens the saved signal plots, the *Light explained by licks* figure and the Program timeline as tabs in the Hub's output area, one per DFM and kind.
- **Open qc folder** shows the member's `qc/` folder in the system file browser.

Where QC runs

- **Hub → QC → Run QC** writes the ticked items, as above.

- **Hub → Analyze → Run analysis** writes auto-removal, `summary.txt` (with the QC summary) and the feeding summaries, then the ticked analyses, among them the light table. It writes nothing into `qc/`.
- **Hub → Project → Analyze all** writes, for every member, the QC bundle, Basic analysis (the optogenetic light QC, the light table, the Program timeline of each firmware 4.0.0 DFM of an optogenetic member, a Progressive Ratio experiment's own light QC) and the report, **whatever the panels' ticks**. A Project Script's `run_all_analyses` step, in a Batch Run or not, runs the same Basic analysis (the report only when the step asks for it) and does not read the ticks either.
- **The QC Viewer**: loading a member on its **Load** tab writes the QC bundle. Opened from the Hub, the viewer starts on the member the Hub already loaded without writing anything, and **Run QC** in its top bar writes the whole bundle, whatever the Hub's ticks.
- **The experiment report** writes nothing into `qc/`. It computes its integrity and crosstalk tables from the loaded recording, never from an old `qc/` folder.
- **Scripts**: `run_qc` writes the ticked items, as the Hub's Run QC does. `basic_analysis` leaves the QC bundle out, but still writes the optogenetic light QC into `qc/opto/` and the Program timeline of each firmware 4.0.0 DFM, as Basic analysis always does.

From Python, `execute_basic_analysis()` writes the QC bundle too, unless you pass `skip_qc=True`.

The QC bundle describes the detection parameters in force when it was written. **Write it again after you change a parameter**, or the baselined plot will show thresholds you no longer use.

What goes where

```
my_experiment/
  remove_chambers.csv        exclusions you declare (section 5)
  qc/
    integrity/               DFM1_integrity_report.csv and .txt
    data_breaks/             DFM1_data_breaks.csv, only when there are breaks
    raw_signal/              DFM1_raw.png
    baselined/               DFM1_baselined.png
    cumulative_licks/        DFM1_cumulative_licks.png
    simultaneous_feeding/    DFM1_simultaneous_feeding_matrix.csv   (two-well)
    bleeding/                DFM1_bleeding_matrix.csv
                             DFM1_bleeding_alldata.csv               (two-well)
    opto/                    four tables and opto_light_dfm1.png     (optogenetic)
    program/                 program_timeline_dfm1.png               (optogenetic)
  analysis/
    removed_chambers.csv     what auto-removal took out, and why
    summary.txt              the run in text, with its QC sections
    experiment_report.pdf    the report, which opens with Quality control
    light_summary.csv, ...   the light table                         (optogenetic)
    pr_light_qc.csv, ...     Progressive Ratio (section 8)
```

Every file outside `qc/opto/` and `qc/program/` has one copy per DFM, named `DFM<n>_...`.

`summary.txt` always lists the chambers excluded and the chambers auto-removal took out, and it has a light QC section for an optogenetic or Progressive Ratio experiment. Its *QC summary* of integrity, data breaks and crosstalk appears when the QC ran as part of the analysis: from Python by default, from the Project's **Analyze**

all, and from the Hub's **Run analysis** (and a script's `run_analysis`), which computes it for `summary.txt`. A script's `basic_analysis` leaves that section out; the report and `qc/` hold the same information.

---

3. The checks every experiment gets

3.1 Data integrity

For each DFM, the integrity report states:

Item	What it is
Generation	which shape of data file the DFM wrote: 1, 2 or 3 (section 6.2)
Rows	the number of samples recorded
Start, end and elapsed time	the first and last sample's clock time, and the minutes between them
Elapsed from the `Minutes` column	the same span, from the file's own minute count
Error column	how many samples carry a non-zero firmware error code, and how many of each kind (below)
Index check	whether the device's record index rises by exactly 1 from each row to the next
Lost records	records missing from the index: how many, and for each gap the index before it, its minute and its length in seconds
Repeated records	records written twice, and dropped at load
Paradigm state	generation 3 only: the baseline rows before the program's first instruction (the lead-in: Program Interval 0, `Paradigm` `NA`), the rows under an instruction outside the program, the late records, and the Paradigm sequence — each Paradigm with its first and last minute and its rows
DFM restarts	generation 3 only: each time the DFM restarted on its own during the recording (below)
`Messages.csv` errors	the error messages the MCU logged for this DFM and for itself, when `data/` holds its `Messages.csv`; on generation 3, its lost-record count checked against the data

The error kinds depend on the generation. The firmware's error byte has eight bits, and the meaning of the two lowest changed:

Bit	Generation 1	Generations 2 and 3
0x01 and 0x02	counted together as *No ACK / overflow*: the two firmware 2.0.0 builds used them differently, and the file does not say which wrote it	0x01 *No Ack*: a request came before the last reply was acknowledged, and records were sent again; 0x02 *Overflow*: the DFM's record buffer overflowed and records were lost

Bit	Generation 1	Generations 2 and 3
0x04 to 0x80	PacketType, DMA_TX, DMA_RX, PacketSize, AllInterrupt and OInterrupt Error	the same

Lost and repeated records. The DFM numbers its records from 1, one per 200 ms. A gap in that index is records the DFM could not keep while the MCU was not reading it; they are gone, and PyMCU logs *Lost N records* in `Messages.csv` (*Skipped index* in builds before 10/17/2026). A repeated index with the same time is a block PyMCU wrote twice after a failed file write: pyflic keeps the first copy, drops the second at load, and `summary.txt` and the integrity report say how many.

DFM restarts (firmware 4.0.0). A DFM can restart on its own in the middle of a recording, after a power dip or a loose cable, say. PyMCU keeps `Index` and the times running across the outage, so the records the DFM did not make are a gap in `Index`. A few rows under no instruction (`Interval NA`) follow, until PyMCU has sent the instruction again, and from there the DFM runs its Paradigm afresh: Acclimation, the Requirement series and the count of Main Activations start over, while the Activation totals carry on. pyflic finds a restart in the data as exactly that: an `Index` gap followed by rows under no instruction. The integrity text's *DFM restarts* section lists each one with its clock time and minute, the records and seconds missing, the rows under no instruction, and whether `Messages.csv` names it (PyMCU writes "DFM n restarted: …" and "DFM n recorded nothing for about S s while it restarted: …"). A restart that only `Messages.csv` names is listed too, with a note saying what the data show there. The light QC warns about every restart (*DFM restarted*, section 7.6), and the Program timeline marks it (section 6.8).

The integrity report's *Lost records* still counts every gap, a restart's among them. The light QC's own *lost records* check leaves a restart's gap out, because *DFM restarted* reports it.

Two more lines on a firmware 4.0.0 recording.

- **The No Ack note.** When *No Ack* is counted with nothing lost or repeated, the error section adds: *No Ack with nothing lost or repeated: PyMCU builds before 10/17/2026 sent a partial ACK the firmware took as missed. No data were lost.* Nothing needs doing.
- **The lost-records cross-check.** With a `Messages.csv`, the records it says were lost are added up and compared with the `Index` gaps outside restarts: *Lost records Messages.csv names: 12, as the Index gaps outside restarts show*, or a sentence giving the difference. A difference means the file and the data disagree about what went missing; the data's gaps are what the analysis stands on.

Where to look. `qc/integrity/DFM<n>_integrity_report.txt` holds the full text and the `.csv` beside it one row of numbers, the generation and the lost, repeated, unknown-instruction and late records among them. The QC Viewer's **Integrity** tab shows both, with the recording's `Messages.csv` below. The report's *Data integrity* table gives each DFM a verdict: **ok** when it has no data breaks, no error flags, no lost records and a continuous index, and **warning** otherwise. The report's cover repeats the verdict under *At a glance*.

Reading it.

- The two elapsed times should agree with each other and with how long you recorded. A DFM much shorter than the others stopped early.
- An index that does not rise by 1 means records are missing or repeated. Lost records leave a data break at the same minute (section 3.2).

- The error names are the firmware's. pyflic counts them but does not interpret them. *Overflow* beside lost records is the same event seen twice.
- **Nothing here removes a chamber.** A warning is a reason to look at the signal plots around the affected time, and to note it if that DFM's results stand out. On an optogenetic recording the light QC also warns about lost, late and unknown-instruction records (section 6.3) and about restarts (section 7.6).

3.2 Data breaks

A **data break** is a gap between two consecutive samples longer than four sampling intervals: more than 0.8 seconds at the default 5 samples per second. `qc/data_breaks/DFM<n>_data_breaks.csv` has one row per break: the row that ended it, led by `BreakIndex`, its row number in the recording, and `GapSeconds`, the length of the gap. The file exists only for a DFM that has breaks. The report's integrity table counts them. (Older pyflic named these two columns `Index` and `Interval`, which clashed with the data file's own columns and stopped the check on recordings that had a break.)

- **A few short breaks change little.** Totals lose a second or two.
- **A long break removes that stretch from every chamber on the DFM.** Compare the DFM's recording length with the others', and treat time-binned results across the gap with care.
- **Breaks on one DFM only** point at that device or its connection, not at the flies.

3.3 The signal plots

Three figures per DFM, one panel per well. They are diagnostics, not figures for a paper.

Raw signal (`qc/raw_signal/DFM<n>_raw.png`). Every well's signal as recorded.

- A well that stays flat, with no deflections at all, is an empty position or a dead well.
- A background that creeps upward is drift. The baseline removes slow drift, as the next figure shows, but a well that drifts a lot is worth a second look.
- A sudden step in the background is a disturbance: a moved plate, a refilled well, a bubble.

Baselined signal (`qc/baselined/DFM<n>_baselined.png`). Every well after pyflic subtracts a running median of the signal, its **baseline**, with two red dashed lines: `feeding_threshold` (the upper line) and `feeding_minimum` (the lower one). A feeding bout must reach the upper line, and its licks are the samples above the lower one.

- **This is the most useful figure in the set.** It shows whether the thresholds sit in a sensible place for your data.
- The trace should sit near zero between bouts. A trace that dips below zero around long bouts, or long bouts that look flattened, means the baseline is being pulled up by the feeding itself; see *The raw signal and the baseline* in the help.
- Bouts that barely cross the upper line, or noise that often crosses it, both say the threshold needs a look. Try values in the QC Viewer (section 4) before changing the configuration.

Cumulative licks (`qc/cumulative_licks/DFM<n>_cumulative_licks.png`). Each well's running lick total over time, drawn as the fourth root of the total whatever `transform_licks` says. A two-well experiment has one row per chamber, with well A on the left and well B on the right.

- A steady slope is steady feeding, a plateau is a fly that stopped, and a step is a burst.

- A curve that never leaves zero is a well with no licks. With a fly in the chamber, that fly did not feed there, or the well did not work.
- One chamber far steeper than its neighbours is either a heavy feeder or a signal problem. The baselined plot tells you which.

3.4 Simultaneous feeding (two-well)

A fly cannot drink from both wells of its chamber at once. Samples that register a lick on both wells are therefore **crosstalk** between the wells, or a fly bridging them. `qc/simultaneous_feeding/DFM<n>_simultaneous_feeding_matrix.csv` has one row per chamber:

Column	Meaning
`Licks1`, `Licks2`	lick samples at the chamber's lower- and higher-numbered well, after the correction
`Both`	lick samples on both wells at once, after the correction
`MaxMinSignalAtBoth`	across those samples, the largest value of the weaker well's signal
`HigherInCol1AtBoth`	how many of them had the lower-numbered well's signal higher
`Both_pre`, `MaxMinSignalAtBoth_pre`, `HigherInCol1AtBoth_pre`	the same three, before the correction

The QC Viewer's **Sim. Feeding** tab shows the table, and the report's *Simultaneous feeding* table gives each chamber's treatment, the lick samples at each well, the samples on both wells **before** and **after** the correction, and the count after it as a percentage of the chamber's lick samples.

The dual-feeding correction. `correct_for_dual_feeding`, on by default in two-well experiments, gives each such sample to the stronger well and detects the licks again. The `_pre` columns are the crosstalk the detection found; the columns without them are what the correction left, normally zero. The lick counts are the ones each well kept. With the correction off the two agree. State in your methods whether the correction was on.

Crosstalk left uncorrected pushes a chamber's preference index toward zero, because the weaker well is credited with licks that belong to the stronger one. The correction repairs most of that, but a chamber where crosstalk is common deserves a look at its baselined trace.

3.5 Bleeding between wells (two-well)

Bleeding is one well's signal appearing in another well. For every well in turn, pyflic takes the samples where that well's baselined signal is above 50 and averages every well's baselined signal over them. The result is a 12 × 12 table in `qc/bleeding/DFM<n>_bleeding_matrix.csv`: a row per signalling well (`W1Sig` …) and a column per responding well (`W1Resp` …). `DFM<n>_bleeding_alldata.csv` is each well's mean baselined signal over the whole recording, for comparison.

- The diagonal is each well responding to itself, which is large by construction. Ignore it.

- **Off the diagonal, every value should be near zero.** A well that rises while another is being fed from is picking up its signal.
- **The report's check.** For each DFM, the report's *Bleeding between wells* table names the largest response off the diagonal. It is a **warning** when that response is above `feeding_threshold`, because such a response would register as licks that never happened.

A bleeding warning is a reason to open the baselined plot of the responding well and look at the moments the signalling well was fed from. Nothing is removed automatically.

---

4. The QC Viewer

The QC Viewer is where you check detection against the trace. Open it with **Open QC Viewer** on the **View** card of the Hub's **QC** panel, which hands it the member the Hub has loaded, or from a terminal:

```
pyflic qc "path/to/my_experiment"
```

Tab	Shows
Load	load an experiment and choose the time range; the log of every step
Feeding Summary	the per-chamber summary table, with a checkbox per chamber for exclusion
DFM *n*	one tab per device: **Integrity** (with the recording's `Messages.csv` below), **Sim. Feeding** and **Bleeding** (two-well), **Raw Signal**, **Baselined** and **Cumulative Licks**, and **Program** for a DFM with a program or firmware 4.0.0 data (section 6.8), with its *Wells* and, on firmware 4.0.0, *Activations* tables (section 7.7); an **Exclude Wells** panel sits beside them
Opto Light QC	optogenetic experiments only (sections 6.6 and 7.7)
Params	the detection parameters, with a live recompute

What to look for

The DFM tabs show the saved QC plots of section 3.3. They do not mark individual events. Judge detection by setting a well's **Baselined** trace, with its threshold lines, beside its **Cumulative Licks** curve, which climbs only where pyflic detected licks.

- **Does the baseline sit where it should?** Between bouts the baselined trace should sit near zero.
- **Does detection match the trace?** Deflections that clear the upper line should appear as climbs in the cumulative curve. Climbs where the trace is flat mean the thresholds are too low. Clear deflections with no climb mean they are too high.
- **Are bouts split or merged?** The plots cannot show this, but the Feeding Summary can. Many short events per chamber suggest the link gap (`feeding_event_link_gap`) is too small, and a few very long ones that it is too large. This is the most common cause of surprising duration and event-count results.
- **Are some wells dead?** A well with no signal is an empty position or a hardware fault. Exclude it rather than let a zero into the averages.

Trying parameters

The **Params** tab changes a detection parameter and recomputes the feeding summary, without editing your configuration. Change one value, recompute, look at a chamber you can read by eye, and repeat. When detection matches your reading of the trace, write the value into the configuration. The Params tab never saves it for you.

A recompute updates the Feeding Summary and the Opto Light QC tabs, and each DFM tab's Program sub-tab. The DFM tabs' other plots are the images saved in `qc/`, which do not change. **Run QC** after a recompute writes the QC bundle under the parameters you are trying, not the ones in your configuration. Once you have settled on a value and written it into the configuration, load the member in the Hub again (double-click its row) and **Run QC** there, so the QC bundle matches what the analysis uses.

Before you publish, sweep the parameter. The Script Editor's *Parameter sensitivity sweep* (`param_sensitivity`) re-runs the analysis over a list of values and writes `analysis/param_sensitivity_<parameter>.csv`, with the licks, events and median duration of each treatment at each value. An effect that appears at only one link gap or one threshold is not a robust effect.

Marking exclusions

On the **Feeding Summary** tab, tick the chambers to exclude, or use a DFM tab's **Exclude Wells** panel, which ticks the chamber a well belongs to. **Mark All Excluded** and **Clear All** set every box at once. **Auto Filter** ticks the chambers the design's cutoffs would remove, and **View Criteria** shows the rules it used and why each chamber was chosen. **Save Exclusions...** writes your selection to a named group in the member's `remove_chambers.csv`.

The ticks are only a selection until you save them. Review Auto Filter's selection before saving: automatic criteria are a starting point, not a verdict.

---

5. Exclusions

An **excluded chamber** leaves every result: every summary, plot and statistic. Chambers leave in two ways.

By hand

Each member declares its own exclusions in a `remove_chambers.csv` at its root:

```
group,dfm_id,chamber,note
general,1,3,low lick count
general,2,5,fly escaped
```

The `group` column lets one member keep several exclusion sets. The active one is `general`, unless a Project's design names another with `exclusion_group:`. The QC Viewer's **Save Exclusions...** writes this file for you.

To declare many at once, put one exclusion sheet at the project or batch folder and apply it; pyflic writes its rows into each member's file. See *Excluding chambers in bulk* in the help.

Automatically, by the design's cutoffs

Basic analysis, and the Hub's Run analysis, apply the design's `constants:` cutoffs once, before writing any summary. The defaults depend on the experiment type (a type your lab built states its own), and the median-duration and events cutoffs apply to every two-well type that sets them:

Setting	Removes a chamber when	Custom	Hedonic, Progressive Ratio
(always)	a well produced no usable lick count	yes	yes
`min_untransformed_licks_cutoff`	a well has fewer licks than this over the recording, counted before any transform	off	20
`max_med_duration_cutoff`	the median feeding-event duration at either well is above this, a sign of a stuck or bleeding signal (two-well)	off	13.0
`max_events_cutoff`	either well has more feeding events than this (two-well)	off	150000
`exclude_failed_opto_chambers`	its linkage group failed the optogenetic light QC (sections 6 and 7)	off	off

Off means the cutoff runs only when the design sets a value. In the Config Editor and the Project Design dialog each cutoff has a **Use** box: ticked with a blank value, it keeps the type's default; ticked with a number, it uses that number; unticked, it turns the cutoff off even where the type sets one.

A Progressive Ratio experiment adds its own group-level rules (section 8). Set the cutoffs in the Project's design so that every member is judged by the same rule.

Where exclusions are recorded

File	Holds
`remove_chambers.csv`	the exclusions you declared
`analysis/removed_chambers.csv`	every automatic removal and its reason
`analysis/summary.txt`	both lists
the report's *Excluded chambers* table	both, marked *by hand* or *automatic*; the cover counts them
`<project>_Excluded.csv`	every member's, in a Project (section 9)

Declaring does not change results already on disk. When a member's `remove_chambers.csv` is newer than its saved results, the Hub's Analyzed column reads **re-run needed**. Run the member's analysis again (**Run analysis**, or the Project's **Analyze all**).

---

6. Optogenetic experiments: the light QC

When a DFM drives an Optolid, the firmware decides every millisecond whether each well's light is on. It reads the well's own signal, compares it with a threshold from the program the MCU loaded, and follows the rule of the interval's **Paradigm**: in closed loop, for example, it keeps the light on for a set **Hold** after contact ends. pyflic counts licks afterwards, from the baselined signal.

Usually the two agree. When a well's signal drifts upward, however, the firmware sees a well that is constantly touched and keeps the light on, while pyflic sees a flat trace and no licks. The light then describes the sensor, not the fly. The **optogenetic light QC** measures that disagreement over the whole recording, for any experiment type.

A short tail of light after the last lick is normal: that is the Hold. Long stretches of light with no lick are not.

What pyflic can see depends on the recording. Older firmware records only `OptoCol1`, which LEDs were lit at each sample. DFM firmware 4.0.0 also records, on every row, the Paradigm in force and each well's own decisions: its Activations, its Acclimation, and the milliseconds its light was on. Section 6.2 explains the difference. Because those records time every light period to the millisecond, a firmware 4.0.0 recording also gets the rule checks of section 7: each Paradigm checked against its own rule.

6.1 Turning it on

One setting, `optogenetics`, in the design's `global:` block or the experiment's own:

Value	Effect
`auto` (the default)	runs on every DFM that has a section in `Program.txt` or any light on in its data
`true` (*yes* in the editors)	runs on every DFM. A DFM with no light on for the whole recording **fails**: an unplugged lid, or a program that did not load
`false` (*no*)	off

A DFM's own entry may override it, because whether its lid was lit is a fact about the recording:

```
dfms:
- id: 3
  optogenetics: false     # this DFM ran without a lid
```

In the Config Editor the setting is on the Experiment tab, and each DFM tab has its own *Optogenetics* picker. In the Project Design dialog it is a design setting.

6.2 Program.txt and the generations

The MCU writes the program it ran back out as `Program.txt`. **Copy that file into the member's `data/` folder**, beside the DFM files. Use the exported file, the one with `***DFM 1***` sections, not the program you wrote for the MCU.

Two exports. PyMCU 4.0.0 writes a new kind; older PyMCU wrote another. pyflic tells them apart by their content and reads both.

- **The PyMCU 4.0.0 export** opens with a `Paradigms:` block that names every **Paradigm**, its type and every setting:

  ```
  PR10 (ProgressiveRatio): Frequency 40 Hz, PulseWidth 8 ms, ActivationMs 500 ms, Series Arithmetic, FirstRequirement 1, Step 10, Cap 200, EventGapMs 100 ms, Acclimation 10
  ```

  Each interval line then names its Paradigm, with a threshold per well that is a level, `On` or `Off`:

  ```
  (10/12/2026 20:30:00) PR10, Dark, 20,Off,20,Off,20,Off,20,Off,20,Off,20,Off, 330 min
  ```

  Earlier 4.0.0 exports put `Dark` or `Lit` before the Paradigm's name; pyflic reads both orders.

  An old program loaded into PyMCU 4.0.0 appears with generated names such as `ClosedLoop-40Hz-8ms-Hold500-MaxOn0`. The `***Env Mon***` section, the Environmental Monitor's, is ignored.

- **The older export** echoes five numbers on each interval line: `F:`, `P:`, `D:`, `L:` and `M:`, frequency, pulse width, decay, delay and maximum time on. Thresholds are numbers: `0` is open loop and `-1` never triggers. pyflic turns each line's numbers into the Paradigm PyMCU would make of them; the decay becomes its Hold.

- **A recording started without a Program** exports one `***ALL DFM***` block. pyflic reads it as every well `Off` for the whole recording, so any light is a fault.

From either, pyflic learns for each DFM and each interval of the program:

- **each well's threshold.** A level makes the well a **trigger well**. `On` keeps its light on regardless (open loop), and `Off` means it never lights by its own behaviour;
- **the linkage**: wells that share a linkage number form one **linkage group**, lit together whenever any of them triggers;
- **the Paradigm**: its type (OpenLoop, ClosedLoop, FixedInterval, NonFeeding, ProgressiveRatio or FixedRatio) and settings, and from them the **Hold** — `HoldMs` for ClosedLoop, `ActivationMs` for FixedInterval and the two ratio types;
- **the schedule**: the program type and the interval timestamps.

The generations. No data file states its firmware version, so pyflic reads the shape of each DFM's file once, at load:

Generation	Recorded by	What pyflic can read
1	PyMCU 3.x, older firmware: 27 columns	the light at each sample (`OptoCol1`); Training as a flag in the signal
2	PyMCU 4.0.0 with an un-updated DFM: 90	the light at each sample; no Training at all

Generation	Recorded by	What pyflic can read
	columns, the new ones empty	
3	PyMCU 4.0.0 with DFM firmware 4.0.0: 90 columns, filled	on every row the Program Interval, its Cycle and its Paradigm; for every well its Phase (`A` in Acclimation, `M` after), its Activations, its Requirement and Progress under a ratio Paradigm, and `OnMs`, the milliseconds its light was on

On generation 3 the rows are the schedule, so the program's timestamps are only a check (*interval started late*, section 7.6), and the light QC reads the firmware's decisions instead of re-creating them. The integrity report, `summary.txt` and the report's cover all name each DFM's generation.

The file is optional, but the check is weaker without it.

- On generations 1 and 2 pyflic infers the linkage from wells lit together, treats every lit well as a trigger well, and assumes a Hold of `opto_default_decay_ms`.
- On generation 3 the rows name each Paradigm but not its type, settings or thresholds. pyflic takes the linkage from wells whose light matches to the millisecond and the trigger wells from the Activations, judges every Activation against its group's licks within `opto_default_decay_ms`, and leaves every type-dependent result `NA`. None of section 7's rule checks runs, because each needs its Paradigm's settings. Because the file was simply not copied, this is a loud warning: `pyflic lint`, the report's *At a glance* and the Hub's **Loaded** line say *DFM n: firmware 4.0.0 recording without Program.txt: Paradigm types and settings unknown*, and the light QC flags *no Program.txt for firmware 4.0.0*.

Either way unexplained light can then warn but never fail, because without the program an open-loop schedule looks exactly like a stuck light. `summary.txt` and the report say so.

If the file cannot be read in part, the lines pyflic skipped are listed in `summary.txt`, and a DFM whose section cannot be read is judged as if it had no program. Two `Program.txt` files in `data/` stop the load: keep only the one the MCU exported for this recording.

The program is checked against the Experiment Type. Each type states the light it expects: Hedonic none, ProgressiveRatio its two ratio Paradigms (with open-loop and dark intervals beside them), Custom anything, and a lab's own type what it says. A `Program.txt` that runs any other Paradigm type stops the load, naming the Paradigm type, the DFM and the Program Interval: choose a type that expects that light, or build one in the Experiment Type Editor. This check only says what should have run. How the light QC judges the light always follows the Paradigms that ran, whatever the type.

6.3 What it checks

The unit of judgement is the **linkage group**, because its light is one circuit. Every verdict passes to each chamber the group touches. Each group is judged interval by interval, under that interval's Paradigm: on generations 1 and 2 the intervals are laid out by clock time from the program, and on generation 3 each run of rows under one Program Interval and Cycle is an interval.

Explained light. Light is **explained** when one of its group's trigger wells had a feeding lick, or a tasting-level touch, behind it. Touches count because the firmware decides on contact at millisecond resolution, and a brief

touch it acts on often reads below pyflic's feeding threshold. What "behind it" means depends on the Paradigm type:

Light under	Is explained when
ClosedLoop	a lick or touch at a trigger well from the Hold before it until just after it
FixedInterval, **ProgressiveRatio**, **FixedRatio**	the trigger well had a lick or touch at the Activation's onset: then its whole `ActivationMs` of light is explained. Feeding during a FixedInterval lockout, which earns no light, is expected
Acclimation	the ClosedLoop rule, with `ActivationMs` as the Hold
NonFeeding	not judged this way: the light is meant to be on while the fly does *not* feed. Firmware 4.0.0 recordings are checked for light that stayed on through feeding instead, and for light that stayed off when the fly did not feed (section 7.5)
OpenLoop	not judged this way: open-loop intervals get their lit-fraction check

Two samples of slack each side (`opto_decay_tolerance_samples`) absorb the difference in timing between the firmware's decisions and the samples.

On a firmware 4.0.0 recording the light belongs to Activations. Lit time is `OnMs`, exact to the millisecond. Each Activation of a trigger well owns its group's light from its onset to the next Activation, the light going out or the end of its interval plus the Hold. Unexplained light is the lit time of the Activations whose onset had no lick or touch at their well. On older recordings every lit sample of `OptoCol1` is judged with the Hold rule.

Check	What it means	Verdict
Unexplained light	at least 30% of the group's judged lit time had no lick or touch behind it	**fails** the group
Partly unexplained light	at least 10% unexplained	warning
Light while off	lit while every well of the group was `Off` (-1), or while the DFM ran PyMCU's idle Paradigm	**fails** the group
No light recorded	`optogenetics: true`, and no light on the DFM at all	**fails** the DFM
Broken yoking	firmware 4.0.0: the group's linked wells' light differs by more than 1 ms in more than 1% of its lit records, or an `Off` well linked to a trigger well counted Activations of its own	**fails** the group
Paradigm not in program	firmware 4.0.0: a row names a Paradigm `Program.txt` does not define, so the file is another recording's; those rows are not judged	**fails** every group of the DFM
Unlit feeding	at least half of the feeding bouts at its trigger wells under closed loop, or in Acclimation, never lit the group (at least 5 bouts)	warning

Check	What it means	Verdict
No light events	feeding at its trigger wells, and never lit	warning
Open loop not lit	lit for less than 95% of its open-loop time; on firmware 4.0.0, of any one open-loop Program Interval or pass (section 7.2)	warning
Non-feeding light during feeding	firmware 4.0.0: under NonFeeding, lit through more than 10% of the feeding at its trigger wells, and for at least 30 seconds (section 7.5)	warning
Acclimation count mismatch	firmware 4.0.0: a trigger well's completed Acclimation counted a different number of Activations than its Paradigm's `Acclimation`	warning
Firmware events disagree	firmware 4.0.0: under a ratio Paradigm's Main phase, the feeding bouts the firmware counted at a trigger well and the ones pyflic counted differ by more than 25% of the firmware's (judged from 5 bouts)	warning
Lost records	firmware 4.0.0: more than 25 records missing from the DFM's index, not counting the gap of a restart, which *DFM restarted* reports	warning for the DFM
Unknown-instruction rows	firmware 4.0.0: more than 1% of the rows after the lead-in (Program Interval 0, `Paradigm NA`) recorded under an instruction outside the program; a restart's rows under no instruction count among them	warning for the DFM
Late records	firmware 4.0.0: more than 10 records whose light count exceeds the record's 200 ms, so the record came late	warning for the DFM
Program mismatch	the data's frequency, pulse width or dark setting differ from the program, the data start more than a minute from its start time, or (firmware 4.0.0) a row names another Paradigm than the program's line for its interval	warning
No program section	`Program.txt` has no usable section for this DFM	warning
No Program.txt for firmware 4.0.0	a firmware 4.0.0 DFM without its program	warning
The rule checks	firmware 4.0.0 with `Program.txt`: each Paradigm's own rule, one Activation at a time (section 7). *Short activations*, *max on exceeded*, *wrong activation length*, *lockout broken* and *requirement series wrong* fail; *contact without activation*, *non-feeding dark*, *interval started late* and *DFM restarted* warn	**fails** the group, or warning

Neither unexplained-light check is judged until a group has at least 30 seconds of unexplained light (`opto_unexplained_min_sec`), so a few stray samples in a group lit for seconds are not called a fault. The fractions are the same for every Paradigm type.

Some light is not judged at all:

- light at the start of an interval, for one Hold, which belongs to the interval before it;
- light outside the program's schedule; on firmware 4.0.0, the baseline rows before the program's first instruction, rows under an instruction outside the program, and an interval whose Paradigm `Program.txt` does not define;
- **open-loop** light and **NonFeeding** light, where the light is not meant to follow feeding. They get their own checks instead.

The firmware 4.0.0 checks in this table read the file's own bookkeeping. Their thresholds (section 6.5) were set before any real firmware 4.0.0 recording existed, and the rule checks' were calibrated on simulated recordings only (section 7.1). Bench recordings of each Paradigm type will revisit them, so treat a warning from one as a reason to look.

6.4 Drift or hardware?

Each flagged group gets a **likely cause**:

- **a drifting baseline or sustained contact**, when most of its unexplained light fell where the firmware saw contact and pyflic saw no lick or touch. The light followed the sensor.
- **hardware, linkage or program mismatch**, otherwise: light the firmware had no reason to switch on. Check the lid, the linkage and the program.

How pyflic knows where the firmware saw contact depends on the generation.

On a firmware 4.0.0 recording the firmware's decisions are recorded. An Activation, or a feeding bout the firmware counted toward a Requirement, with no pyflic lick or touch at its trigger well is the drifting-baseline signature itself. In `opto_light_qc.csv`, `ContactSec` is the lit time of the trigger wells' Activations, `ContactWithoutActivitySec` the lit time of the ones whose onset had no lick or touch, and `FirmwareEventsWithoutActivity` the firmware's feeding bouts with none either. Nothing is emulated.

On older recordings pyflic emulates the firmware. With a program, it re-runs the firmware's own test on the recorded signal: the firmware subtracts the mean of the run's first ten seconds and compares each sample with the interval's threshold. Wherever this **emulated trigger** reads a trigger well above threshold while pyflic sees no lick or touch, the firmware was seeing contact that pyflic's running baseline removed. `ContactWithoutActivitySec` is how long that lasted. The emulation is marked approximate (`EmulationApproximate`) when the data begin more than two seconds after the program started, because the firmware's first ten seconds were not recorded.

6.5 Settings

The thresholds are design `constants:`, shared by every experiment type, with fields in the Project Design dialog and the Config Editor. Change one only for a reason you can state.

Setting	Default	Meaning
`exclude_failed_opto_chambers`	false	remove every chamber of a failed linkage group; off, they are flagged and kept
`opto_unexplained_warn_fraction`	0.10	unexplained share of lit time that warns
`opto_unexplained_fail_fraction`	0.30	unexplained share that fails

Setting	Default	Meaning
`opto_unexplained_min_sec`	30	seconds of unexplained light needed before either is judged
`opto_default_decay_ms`	1000	the Hold assumed without `Program.txt`
`opto_decay_tolerance_samples`	2	samples of slack around the Hold
`opto_unlit_feeding_fraction`	0.50	unlit share of closed-loop feeding bouts that warns
`opto_open_loop_min_lit_fraction`	0.95	lit share an open-loop interval needs
`opto_event_disagreement_warn_fraction`	0.25	firmware 4.0.0: a trigger well whose firmware and pyflic feeding-bout counts differ by more than this share of the firmware's warns
`opto_yoking_fail_fraction`	0.01	firmware 4.0.0: share of a group's lit records in which its linked wells' light differs by more than 1 ms, above which the group fails
`opto_lost_records_warn`	25	firmware 4.0.0: records missing from the index (5 seconds) above which the DFM warns
`opto_unknown_rows_warn_fraction`	0.01	firmware 4.0.0: share of rows under an instruction outside the program above which the DFM warns
`opto_late_records_warn`	10	firmware 4.0.0: late records above which the DFM warns
`opto_activation_tolerance_ms`	25	firmware 4.0.0: how far a light period may differ from its Hold, `ActivationMs` or `MaxOnMs`, or two onsets from the lockout, and still keep the rule; one record (200 ms) more for a period timed only to the record
`opto_activation_length_fail_fraction`	0.05	firmware 4.0.0: share of judged Activations breaking their length rule (*wrong activation length*, *short activations*) above which the group fails
`opto_lockout_fail_fraction`	0.02	firmware 4.0.0: share of judged FixedInterval onsets closer than the lockout (*lockout broken*) above which the group fails
`opto_missed_contact_warn_fraction`	0.25	firmware 4.0.0: share of free FixedInterval contact runs that brought no Activation (*contact without activation*) above which the group warns
`opto_non_feeding_lit_warn_fraction`	0.10	firmware 4.0.0: share of a NonFeeding group's feeding during which it was lit (*non-feeding*

Setting	Default	Meaning
		light during feeding) above which it warns
`opto_non_feeding_min_sec`	30	firmware 4.0.0: seconds of that light needed before it is judged
`opto_non_feeding_dark_warn_fraction`	0.10	firmware 4.0.0: share of the time a NonFeeding rule says lit that the light was dark (*non-feeding dark*) above which the group warns
`opto_interval_lag_warn_sec`	5	firmware 4.0.0: seconds a Program Interval may begin after its `Program.txt` time (*interval started late*) before the DFM warns

The last thirteen apply only to firmware 4.0.0 recordings. The five that read the file's bookkeeping (`opto_event_disagreement_warn_fraction` to `opto_late_records_warn`) were set before any real recording existed. The last eight belong to the rule checks of section 7, which describes what each one decides; how they were calibrated is in section 7.1. All of them will be revisited once bench recordings of each Paradigm type exist.

Non-feeding light during feeding used to borrow `opto_unexplained_warn_fraction` and `opto_unexplained_min_sec`. It now reads `opto_non_feeding_lit_warn_fraction` and `opto_non_feeding_min_sec`, whose defaults are the values it borrowed, so its results are unchanged. If your design changed the unexplained-light settings to tune this check, set the new two instead.

6.6 Where to look

Basic analysis and the Hub's **Run QC** (with *Opto light QC tables* and *Light explained by licks* ticked) both write the light QC into `qc/opto/`:

File	One row per	Holds
`opto_light_qc.csv`	linkage group	its `Generation`; its wells, trigger wells, chambers, their `Roles` and treatment; lit and unexplained time; the onset; the contact evidence (section 6.4); on firmware 4.0.0, the firmware's and pyflic's feeding-bout counts, the Acclimation counted and expected, and a column per rule check (section 7.7); `LikelyCause`, `Flags`, `Verdict`, `Excluded`, `Notes`
`opto_light_intervals.csv`	group and interval	the interval's mode, the Hold it was judged with (`HoldMs`) and trigger wells; on firmware 4.0.0 the `ProgramInterval`, `Cycle`, `Paradigm` and `ParadigmType` the rows recorded; lit, unexplained and contact time; feeding bouts, and those never lit
`opto_light_events.csv`	light event	its well, onset, end and duration; unexplained seconds; `OverrunSec`, the light's time past the last lick plus the Hold; on firmware 4.0.0, where each event is one Activation, its

File	One row per	Holds
		`Phase`, the Requirement it met, the firmware's feeding bouts behind it, its light to the millisecond, its Paradigm Family and what ended it (section 7.7)
`opto_program.csv`	program interval	the program as pyflic read it: each Paradigm's name, type and settings, the linkage and trigger wells, and for an older export the raw `F` and `P` it echoed
`opto_light_dfm<n>.png`	DFM	the *Light explained by licks* figure

Roles. `Roles` names what each chamber of the group was to the light, as in `1 Paired, 2 Yoked`. The six Roles are Paired (it holds the group's only trigger well), Yoked (lit by a partner's trigger well), Mutual (one of several chambers with trigger wells in one group), Self (a trigger well whose group lights no other chamber), Open (lit regardless) and Unlit. They come from the program, or on a firmware 4.0.0 recording from the data; an older recording without a program has none.

Every chamber's rows in the feeding summaries carry `OptoLightQC`, its group's flags (empty when the group is ok), and `OptoUnexplainedFraction`, and on an optogenetic member its `Role`, `LitSec` and `Activations`. `summary.txt` has an *Optogenetic light QC* section, and the report has one in Quality control, with the program, the verdicts and the figure, and on firmware 4.0.0 a *Rule check* subsection per Paradigm (section 7.7).

Beside the QC, basic analysis, and the Hub's Run analysis with *Light tables* ticked, write the **light table**, `analysis/light_summary.csv` (per chamber) and `light_wells.csv` (per well): what the light did, with the feeding summary's treatment and factor columns, and on firmware 4.0.0 each Paradigm Family's own measures. It is a result, not a check; the report's *Light behaviour* section compares treatments on it, on firmware 4.0.0 one subsection per family. See *Optogenetic experiments* in the help.

On the Hub, the QC panel shows an **Optogenetics** group whenever the loaded member is optogenetic, and **Run QC** writes what is ticked in it: **Opto light QC tables** writes the tables and logs every flagged group, **Light explained by licks** draws the figure, and **Program timeline** draws section 6.8's figure (greyed when no DFM has one to draw). The light table is the Analyze panel's **Light tables**.

In the QC Viewer, the **Opto Light QC** tab lists every linkage group with its chambers' Roles, lit time, unexplained share, onset, contact with no lick and verdict, and on firmware 4.0.0 the rule checks' columns (section 7.7), and opens on the worst group. Selecting a group shows why it got its verdict, its intervals, light events and program, and its panels of the figure; a column empty on every row is hidden. The tab is computed from the loaded experiment, so a **Params** recompute updates its verdicts at once. That is worth seeing before you settle on a threshold: one that loses real licks makes healthy light look unexplained.

6.7 Reading *Light explained by licks*

One row per linkage group that was ever lit or touched, or was flagged, with the group's wells and verdict in each panel's strip. Basic analysis, the Hub's Run QC and the QC Viewer draw it in 10-minute bins.

- **The left panel** is the group's lit time per bin, stacked: **grey-blue** where a trigger-well lick or touch explains it, **red** where nothing does, **purple** where the group was lit while every threshold was `Off` (-1),

and **light grey** where the light is not judged by licks (open loop, NonFeeding). On a firmware 4.0.0 recording the lit time is `OnMs`.
- **The right panel** is **orange**. On an older recording, when `Program.txt` allows the emulated trigger, it is the time the firmware would have read a trigger well above threshold while pyflic saw no lick or touch. On a firmware 4.0.0 recording, titled *firmware events, no lick*, it is the light of the Activations the firmware counted with no lick or touch at their onset.

How to read a row:

- **Mostly grey-blue**: the light followed the fly. A little red beside it is light that outlasted the last lick or touch pyflic saw by more than the Hold. Small amounts are normal, which is why nothing is judged below 30 seconds or the warning fraction.
- **Red beside orange** in the same bins: a drifting well. The firmware saw contact, pyflic did not, and the light is not evidence of feeding.
- **Red with no orange**: light the firmware had no reason to switch on. Check the lid, the linkage and the program.
- **Purple** anywhere: the program said the light was off. That is a hardware or program problem, whatever the licks did.

6.8 Reading the Program timeline

The **Program timeline** (`qc/program/program_timeline_dfm<n>.png`) shows, for one DFM, what its program ran and what each well did. Run QC, and the QC bundle of basic analysis, draw it for every DFM with a `Program.txt` section or firmware 4.0.0 data; basic analysis without its QC bundle (a script's `basic_analysis`) still draws it for every firmware 4.0.0 DFM. All of them draw it in 10-minute bins. It is in the report's Quality control section, behind the QC panel's **Program timeline** box, among the Hub's **View QC plots** tabs, and in the QC Viewer's **Program** sub-tab of each DFM.

- **The band at the top** is the program: one block per Program Interval, coloured by Paradigm type and labelled with the Paradigm's name, and with `C<n>`, the pass, when the program repeats. **Light grey** is the baseline before the first instruction reached the DFM, **hatched white** an instruction outside the program, and **dark grey** a Paradigm whose type `Program.txt` does not give.
- **One lane per well** follows, labelled with its chamber and the chamber's Role. **Pale orange shading** is Acclimation (Phase `A`), each **black tick** an Activation, and the **blue strip** the share of each bin the light was on. On a firmware 4.0.0 recording a lane is also **hatched** where `Program.txt` sets the well `On` and **grey** where it sets it `Off`, Program Interval by Program Interval.
- **A red band labelled *restart*** crosses the band and the lanes wherever the DFM restarted (firmware 4.0.0), from the last record before the gap to the first row under the instruction again.

How to read it:

- **The band should be the program you meant to run.** A hatched block after the start is the DFM running something outside the program. A dark-grey block is a Paradigm the `Program.txt` in `data/` does not know: it is most likely another recording's file.
- **Each trigger well should leave Acclimation** within the interval that started it. A well still shaded at the end never completed it.
- **Ticks should come where the fly fed.** Under a ratio Paradigm they thin out as the Requirement rises. A dense comb of ticks while the fly's cumulative licks are flat is the light following the sensor.

- **A yoked well has no ticks** and the same blue strip as its partner. Ticks on a yoked well, or a strip that differs, is broken yoking.
- **A hatched (`On`) lane should be lit throughout**, and a grey (`Off`) lane lit only when its linkage group is. A blue strip with gaps in a hatched stretch is the light *open loop not lit* warns about.
- **After a red restart band** the program starts over: a trigger well may go back into Acclimation, and under a ratio Paradigm the Requirement starts again from the first. That is the DFM doing what it should after a restart, not a fault in the program.

In the QC Viewer the timeline sits above two tables. **Wells** gives each well at the end of the loaded range: its Paradigm, its Phase, the minute its Acclimation ended, its Activations, its Requirement and Progress, and its lit seconds. **Activations**, on a firmware 4.0.0 recording, lists every Activation of the DFM's trigger wells (section 7.7).

On an older recording the band is laid out from the program's clock times and is approximate, and the lanes show the sampled light only: no Acclimation, no ticks, no `On` and `Off` shading and no restart bands.

6.9 Deciding what to exclude

A failed group is flagged, not excluded, by default. A faulty light does not make the feeding record wrong. It makes the light-on numbers wrong, and whether that matters depends on your question.

A rule check that fails is first a question about the file. The firmware keeps each Paradigm's rule to the millisecond, so a group that fails *wrong activation length*, *lockout broken*, *requirement series wrong*, *max on exceeded* or *short activations* most often has the wrong `Program.txt` beside it. Find the one exported with this recording, copy it into `data/`, and run again before you exclude anything. If the file is right, the LED or its record is faulty, and the group is a hardware question like any other failure.

Three ways to act on a failure that stands:

- **Keep the group.** Its flags stay on its chambers' rows, and the Project reports it as *retained*. Say so when you report the result.
- **Exclude it by hand.** In the QC Viewer's Opto Light QC tab, **Mark Selected Group Excluded** or **Mark Failed Groups Excluded** tick its chambers, and **Save Exclusions…** records them. The buttons save nothing on their own.
- **Exclude every failed group automatically**, by setting `exclude_failed_opto_chambers: true` in the design.

`UnexplainedOnsetMin` is the start of the first 30-minute window in which a flagged group's unexplained share reached the warning fraction, in minutes from the start of the recording. It is the latest point at which a cutoff you set by hand could fall, if you analyse only the time before the fault.

Whichever you choose, apply it to every group, whatever its treatment, and decide before you look at the results.

---

7. Firmware 4.0.0: each Paradigm checked against its own rule

Section 6 asks whether the light had licks behind it. A firmware 4.0.0 recording lets pyflic ask a second question: did the light follow the rule of the Paradigm that `Program.txt` says ran? The firmware records every Activation and the milliseconds its light was on, and it keeps each Paradigm's rule exactly: a FixedInterval

Activation lasts its `ActivationMs` to the millisecond, a lockout is never cut short, a progressive ratio's Requirements follow their series. So light that breaks the rule is not the fly's doing. It means the file is not the program that ran, or the LED, or its record of itself, is faulty.

7.1 How the checks judge

When they run. On firmware 4.0.0 (generation 3) recordings with the DFM's section in `Program.txt`, because every check reads its Paradigm's settings. Without the file, or without the DFM's section, they do not run, their columns are `NA`, and the report says that each rule is checked only with `Program.txt`'s settings. Generations 1 and 2 keep the checks of section 6: `OptoCol1`, sampled every 200 ms, cannot time a light period. Their report says so, naming the checks that did not run.

By Paradigm Family. The firmware lights the six Paradigm types by four mechanisms, and pyflic checks each **Paradigm Family** with one set of rules:

- **open loop**: OpenLoop, and any well set `On`. Lit whatever the fly does (section 7.2).
- **closed loop**: ClosedLoop, and the Acclimation of FixedInterval, ProgressiveRatio and FixedRatio. Contact lights the LED, and it stays lit for the Hold after contact ends (section 7.3).
- **fixed length**: FixedInterval, ProgressiveRatio and FixedRatio after Acclimation, in Main. An earned Activation lasts exactly `ActivationMs`. Its *interval* branch (FixedInterval) is earned by contact once the lockout has passed; its *ratio* branch (the two ratio types) by the Firmware Event that meets the Requirement (section 7.4).
- **non-feeding**: NonFeeding. Dark while the fly feeds (section 7.5).

A well's family follows its Phase: a FixedInterval well in Acclimation is closed loop until its Acclimation ends. Wells set `Off` belong to no family; section 6's *light while off* and *broken yoking* cover them. Two checks apply to every family (section 7.6).

One Activation at a time. Each Activation's light period is read from its first and last records: the record where the Activation was counted says how much of it was lit, and so does the last. Its lit time, `LitMs`, is the sum of `OnMs` over the period. Every Activation is listed, with these times, in `opto_light_events.csv` (section 7.7).

Judged and not judged. A length can be judged only when the light period is the Activation's own and its rule ended it. Two kinds are not judged:

- **merged with other light** (`Overlap`): a linked partner's Activation started inside it, the light was already on when it began, it ran straight into the well's next Activation, two Activations began in one record, or records are missing just before it, so its start is not known;
- **cut short by something else** (`Boundary`): a change of Program Interval or Paradigm, a gap in `Index`, or the end of the data stopped it.

They are counted, and the group's notes say how many: *rule checks: 3 of 512 Activations not judged (2 merged with other light, 1 cut short by an Interval change, a gap in Index or the end of the data)*. Under FixedInterval with a `LockoutMs` shorter than `ActivationMs`, held contact keeps the light on without a break and starts a new Activation every `ActivationMs`, so those periods run into each other and are not judged.

The tolerance. A light period keeps its rule when it is within `opto_activation_tolerance_ms` (default 25 ms) of the setting. A period whose edges are known only to the record, not to the millisecond (`Timing record`), gets one record (200 ms) more.

Per Paradigm. A linkage group's Activations under one Paradigm are pooled over every Program Interval and pass that ran it, and the fractions below are judged on that Paradigm's total. A group that ran two Paradigms of a family is judged on each apart; its columns in `opto_light_qc.csv` add the two up.

Fail or warn. Five checks **fail** the group, because the firmware cannot break these rules with the right program: *short activations*, *max on exceeded*, *wrong activation length*, *lockout broken* and *requirement series wrong*. Their reason names the setting and the value measured, and ends: *the firmware keeps this rule to the millisecond, so Program.txt is probably not the program that ran, or the LED or its OnMs report is faulty*. Four new checks only **warn**: *contact without activation* and *non-feeding dark* rest on pyflic's lick detection, which is not the firmware's threshold, and *interval started late* and *DFM restarted* on the DFM's bookkeeping. Two older warnings, *open loop not lit* and *non-feeding light during feeding*, belong here too, by family. As with every flag, a failure passes to every chamber of the group and removes nothing unless `exclude_failed_opto_chambers` is on.

How the constants were set. The eight constants of these checks (section 6.5) were calibrated on simulated recordings: pyflic's own simulator, which matches the firmware's code millisecond for millisecond, and PyMCU's. On clean data the length, lockout, missed-contact and series counts were 0, *non-feeding dark* reached 2.1% against its 10%, and the worst Interval lag was 0.2 s against 5 s, so no default changed. They will be revisited once bench recordings of each Paradigm type exist.

7.2 Open loop

Open loop not lit (warning)

- **The rule.** Under OpenLoop, and for a well set `On`, the LED is lit for the whole Program Interval, whatever the fly does, and no Activation is counted.
- **What pyflic checks.** The lit fraction of each open-loop window. On a firmware 4.0.0 recording each Program Interval, and each pass of a Repeating program, is a window of its own, and `OpenLoopLitFraction` is the **lowest** of them, so one dark pass shows instead of being averaged away. A window's start, for the Hold of the window before it, is left out, as in section 6.3. This check runs on every generation: on older recordings the group's open-loop time is judged as a whole.
- **Verdict.** A warning when that fraction is below `opto_open_loop_min_lit_fraction` (default 0.95). The reason names the Program Interval and Cycle.
- **Likely causes.** An LED that failed or a lid that came loose for part of the run, or a `Program.txt` that sets a well `On` where the DFM ran something else.
- **What to do.** Find the window in the Program timeline: a gap in the blue strip of a hatched (`On`) lane. Check the lid, and that `Program.txt` is this recording's.

7.3 Closed loop

Short activations (fails)

- **The rule.** A closed-loop Activation is lit while the fly touches the well and for the Hold after contact ends: `HoldMs` under ClosedLoop, `ActivationMs` in Acclimation. Its light can never be shorter than the Hold, or than `MaxOnMs` when that is set and shorter.
- **What pyflic checks.** Each judged ClosedLoop Activation in Main, and each judged Acclimation Activation, against that length less the tolerance. Column `ShortActivations`.
- **Verdict.** Fails the group when more than `opto_activation_length_fail_fraction` (default 0.05) of the judged Activations are short.

- **Likely causes.** The program that ran had a shorter Hold than `Program.txt` states, so the file is another recording's; or the LED, or its `OnMs` report, cuts out.
- **What to do.** Compare the rule figure's lit times with the dashed Hold (section 7.8). Lit times that all begin to the left of the line are a shorter Hold in the program that ran; short periods scattered among healthy ones are the hardware.

Max on exceeded (fails)

- **The rule.** A ClosedLoop Paradigm with `MaxOnMs` above 0 cuts each Activation off at `MaxOnMs`, then keeps the LED dark until the bout ends (`EventGapMs` without contact). NonFeeding with `MaxOnMs` above 0 lights each on-period for at most `MaxOnMs` too (section 7.5).
- **What pyflic checks.** Each judged ClosedLoop Main Activation, and each judged NonFeeding one, against `MaxOnMs` plus the tolerance. Column `OverMaxOnActivations`.
- **Verdict.** Fails the group on any one: the firmware never lets a light period pass `MaxOnMs`.
- **Likely causes.** The program that ran had a longer `MaxOnMs`, or none; or the LED stuck on.
- **What to do.** Check that `Program.txt` is this recording's. The rule figure draws the periods past the dashed `MaxOnMs` line in red.

Cut-offs (reported, no verdict)

- `CutOffs` counts the judged ClosedLoop Main Activations whose light lasted `MaxOnMs`, within the tolerance: the ones the cut-off ended. It describes the fly, not the light: a fly that keeps contact longer than `MaxOnMs` gets cut-offs. The light table carries the same count per chamber.

7.4 Fixed length

Wrong activation length (fails; both branches)

- **The rule.** Under FixedInterval, ProgressiveRatio and FixedRatio, an earned Main Activation lights the LED for exactly `ActivationMs`, whatever the fly does.
- **What pyflic checks.** Each judged Main Activation's lit time against `ActivationMs`, within the tolerance either way. Column `WrongLengthActivations`.
- **Verdict.** Fails the group when more than `opto_activation_length_fail_fraction` (default 0.05) of the judged ones are off.
- **Likely causes.** The program that ran had another `ActivationMs`; or the LED, or its record, is faulty.
- **What to do.** Look at the rule figure's lit-time plot. When the red Activations all sit in one bar away from the dashed line, the program that ran had that `ActivationMs`. Red scattered on both sides is the hardware.

Lockout broken (fails; interval branch)

- **The rule.** Under FixedInterval a contact earns a new Activation only when the last one has ended and `LockoutMs` has passed since it started. Two onsets are never closer than the longer of `LockoutMs` and `ActivationMs`.
- **What pyflic checks.** The time between consecutive Main Activations of one well within one run of the Paradigm (`SinceLastOnsetMs`), against that spacing less the tolerance. Both onsets must be readable, so a pair with a merged period is not judged. Column `LockoutViolations`.
- **Verdict.** Fails the group when more than `opto_lockout_fail_fraction` (default 0.02) of the judged pairs are too close.

- **Likely causes.** The program that ran had a shorter lockout than `Program.txt` states; or the record is faulty.
- **What to do.** The rule figure's *time between onsets* histogram puts the offending pairs in red, left of the dashed spacing. Check the file first.

Contact without activation (warning; interval branch)

- **The rule.** Under FixedInterval, contact lights the LED whenever the well is free: its last Activation is over and the lockout has passed since that one began.
- **What pyflic checks.** Each run of pyflic activity at a trigger well (a feeding lick or a tasting sample) that begins while the well is free, under the Paradigm in Main. It should bring an Activation in its first record or the next. Column `MissedContacts`: the free runs that did not.
- **Verdict.** A warning when more than `opto_missed_contact_warn_fraction` (default 0.25) of the free runs brought no Activation, judged once there are at least 5 free runs.
- **Why only a warning.** The firmware decides on its own threshold: the level `Program.txt` states plus the well's baseline. pyflic's activity is its own detection, and it includes tasting samples, brief touches that can sit below the firmware's threshold. Some missed runs are normal.
- **Likely causes.** A threshold in the program set high for this fly's contact; a resting signal that drifted down, since the firmware measures its threshold from the baseline it took at the start; a sensor that reads low; or a `Program.txt` whose lockout is shorter than the one that ran.
- **What to do.** Look at the trigger well's baselined trace around the missed runs, and at its threshold in `Program.txt`. Brief, low deflections are the threshold; clear licks with no Activation are a fault.

Requirement series wrong (fails; ratio branch)

- **The rule.** Under a ratio Paradigm the k-th Main Activation of a run of the Paradigm meets the k-th Requirement of its series: ProgressiveRatio's Arithmetic (`FirstRequirement`, `Step`) or Exponential (`Scale`, `Rate`) series, capped at its `Cap`; FixedRatio's `Ratio`, every time. A new Paradigm, or a restart, starts the series again at k = 1.
- **What pyflic checks.** Each Main Activation's Requirement met (read from the record before it, as the firmware moves on in the same millisecond) against the series at its number K. A record holding two Activations of one well is not compared. Column `SeriesMismatches`.
- **One case is resolved, not failed.** An instruction that reaches the DFM partway through a record leaves the record's start to the old Paradigm, so an Activation counted on a run's first record may be the old one's. That record shows no Requirement. When every Requirement of the run then matches the series with K one less, the run passes, and a note says so.
- **Verdict.** Fails the group on any one. The reason quotes up to three, as *W3's 4th met 30, the series gives 31 (minute 52.4)*.
- **Likely causes.** The program that ran had another `Step`, `FirstRequirement`, `Scale`, `Rate`, `Cap` or `Ratio`: the file is another recording's.
- **What to do.** The rule figure draws each Requirement met against K over the dashed series; red points are the ones the series does not give. A Progressive Ratio experiment's own light QC fails the chamber group on the same check (section 8).

7.5 Non-feeding

Non-feeding light during feeding (warning)

- **The rule.** Under NonFeeding the LED is dark while the fly touches the well.

- **What pyflic checks.** The share of the feeding at a group's trigger wells during which the group was lit.
- **Verdict.** A warning when that share is above `opto_non_feeding_lit_warn_fraction` (default 0.10), once the group has at least `opto_non_feeding_min_sec` (default 30) seconds of such light. These two are the check's own constants now; their defaults are the unexplained-light values it used to borrow, so its results are unchanged (section 6.5).
- **Likely causes.** A linkage that lights the group from another trigger well, an LED stuck on, or feeding pyflic detects below the firmware's threshold.
- **What to do.** Check the program's `Linkage:` line and the lid; look at the baselined trace of the trigger well.

Non-feeding dark (warning)

- **The rule.** NonFeeding lights the LED `OnDelayMs` after contact ends, keeps it lit for at most `MaxOnMs` (0: until the next contact), and lights it at once when the Paradigm starts, before any contact.
- **What pyflic checks.** From pyflic's activity at each trigger well, the time the rule says the light must be on, and how much of it `OnMs` shows dark. pyflic often misses the last, partial record of a bout, so the expected light starts one record after the last activity, plus `OnDelayMs` and the tolerance. Column `NonFeedingDarkSec`: the seconds dark where the rule says lit.
- **Verdict.** A warning when the light was dark for more than `opto_non_feeding_dark_warn_fraction` (default 0.10) of that time, judged once at least 10 seconds are expected lit.
- **Likely causes.** Contact the firmware read and pyflic did not: a resting signal that drifted up keeps the firmware in contact, so it keeps the light off. Or the program that ran had a longer `OnDelayMs` than `Program.txt` states. This is not a failed lamp: the light pyflic reads is the firmware's own record of driving the LED, so a lamp that fails in the lid does not show in the data at all.
- **What to do.** Look at the Program timeline's blue strip for the trigger wells, and at their raw signal for drift.

Max on exceeded applies here too, when `MaxOnMs` is above 0: one on-period past `MaxOnMs` fails the group (section 7.3).

The relight delay (reported, no verdict)

- `RelightDelayMs` is the median, over the NonFeeding Activations that follow pyflic activity, of the time from the end of the last activity sample to the Activation's onset. It should sit near `OnDelayMs`. pyflic's activity is per record, so a single delay is good to about a record either way.
- No flag watches it. A `Program.txt` whose `OnDelayMs` differs from the program that ran shows only here, as a median well away from the setting, and in the rule figure (section 7.8).

7.6 Every family

Interval started late (warning, for the DFM)

- **The rule.** The DFM takes up a new instruction within a record of PyMCU sending it, so each Program Interval begins, on every pass, at its `Program.txt` time: Linear and Constant programs from the Start Time, Repeating ones a cycle's length later each pass, Circadian ones at their time of day.
- **What pyflic checks.** The first row of each Program Interval, on every Cycle, against that time. The worst lag is `IntervalLagSec`. The DFM's first Interval follows a lead-in: PyMCU sends no instruction until it has taken its baseline, which took 4.8 to 5.2 seconds on the bench and in the simulators, so 5 seconds

of the first lag are discounted. An Interval `Program.txt` does not print (a Circadian day the program started partway through) is not timed.

- **Verdict.** A warning, on every group of the DFM, when a lag is above `opto_interval_lag_warn_sec` (default 5). The reason names the Program Interval, Cycle, lag and minute, and says when the DFM had just come back from a restart.
- **Likely causes.** The DFM did not take up the instruction when it was sent, and PyMCU sent it again; or the start time in `Program.txt` is another recording's.
- **What to do.** Note the minutes. The rows are still judged under the Paradigm they name; the late stretch ran the previous one. Many late Intervals point at the DFM's connection.

DFM restarted (warning, for the DFM)

- **What happened.** The DFM restarted on its own mid-recording (section 3.1): no records for the outage, a few rows under no instruction, then the Paradigm afresh.
- **What pyflic does.** Each restart starts a new run of the Paradigm: Acclimation, the Main Activations' number K and the Requirement series start again after it, as they did on the DFM. The rows under no instruction are not judged. The restart's `Index` gap is not counted toward *lost records*. A Progressive Ratio group whose Test spans a restart has its breaking point judged up to it (section 8). Column `Restarts`, which needs no `Program.txt`.
- **Verdict.** A warning, on every group of the DFM, naming each restart's minute and clock time, the records and seconds missing and where the Paradigm started over.
- **What to do.** Check the DFM's power and cable. Treat results across the restart as two runs of the program: under a Paradigm with Acclimation, a fly that was past it before the restart went through it again after.

7.7 Where to look

`opto_light_qc.csv` gains a column per check, one row per linkage group. Each is `NA` where none of the group's Paradigms has the rule, on generations 1 and 2, and without `Program.txt` (except `Restarts`):

- `ShortActivations`, `OverMaxOnActivations`, `WrongLengthActivations` and `SeriesMismatches`: the judged Activations that broke the rule; `LockoutViolations`: the onset pairs too close together;
- `CutOffs`: the Activations `MaxOnMs` ended;
- `MissedContacts`: the free contact runs that brought no Activation;
- `NonFeedingDarkSec`: seconds dark where the rule says lit;
- `RelightDelayMs`: the median relight delay, in milliseconds;
- `IntervalLagSec`: the DFM's worst Program Interval start, in seconds;
- `Restarts`: the DFM's restarts.

`OpenLoopLitFraction`, there before, is now the lowest over the group's open-loop windows. `Flags` and `Notes` carry the reasons and the not-judged counts.

`opto_light_events.csv` gains, on its firmware 4.0.0 rows (one per Activation), the light period as the checks read it. They are `NA` on rows read from `OptoCol1`.

- `OnsetMs`, `EndMs`, `LitMs`: the light period in milliseconds, counted from the start of the DFM's first record. That origin is a fraction of a second before `OnsetMin`'s, so do not mix the two.
- `Timing`: `exact`, or `record` when the period is good to one record only.
- `Overlap`: true when other light merged with it.

- `Boundary`: `interval`, `gap` or `end` when a Program Interval or Paradigm change, a gap in `Index` or the end of the data stopped it; empty otherwise.
- `SinceLastOnsetMs`: the time since the same well's previous onset in the same run of the Paradigm.
- `K`: its number among the well's Main Activations of that run; empty in Acclimation.
- `Family`: its Paradigm Family (an Acclimation Activation's is closed loop).
- `EndedBy`, what ended it: `Hold` or `MaxOnMs` (closed loop); `length` (fixed length, lasting `ActivationMs`); `contact` or `MaxOnMs` (non-feeding); `interval`, `gap` or `end` (its `Boundary`); `unknown` when it was merged with other light, the Paradigm is not known, or its length is not the rule's.

In the QC Viewer, the **Opto Light QC** tab's group table gains the checks' columns: *Short*, *Over MaxOn*, *Cut-offs*, *Wrong length*, *Lockout broken*, *Missed contacts*, *Series wrong*, *Non-feeding dark (s)*, *Relight (ms)*, *Open loop lit*, *Interval lag (s)* and *Restarts*. A column empty on every row is hidden. Each DFM's **Program** sub-tab has an **Activations** table beside its **Wells** table: every Activation of the DFM's trigger wells, as `opto_light_events.csv` has it, with `Well`, `OnsetMin`, `Family`, `Phase`, `K`, `LitMs`, `EndedBy`, `RequirementMet` and `Overlap`, in order of onset. It is where to find the Activations a check counted: a `LitMs` far from the rule's, an `EndedBy` of `unknown`, a `RequirementMet` off the series.

In the report, the *Optogenetic light QC* section ends with one subsection per Paradigm the program runs, in family order (open loop, closed loop, fixed length, non-feeding), then by name. Each is headed *Rule check:* with the Paradigm, its family (and branch) and its DFMs, as *Rule check: FI10S (fixed length, interval; DFM 1)*. A Paradigm with Acclimation gets two: its Acclimation, under closed loop, and its Main, under its own family. Wells set `On` under another Paradigm get an open-loop subsection of their own, *Wells set On*. Each subsection has:

- **the rule in words, with its settings**, for example *CL500 (closed loop): lit during contact and for 500 ms after; at most 2000 ms per light period, then dark until the bout ends (100 ms without contact)*;
- **a table, one row per linkage group that ran it**: its wells and treatment, the family's columns this Paradigm has, the Activations judged and not judged, and the verdict on this Paradigm alone. The open-loop table gives the lowest lit fraction and the number of windows instead. A group's rows across subsections add up to its `opto_light_qc.csv` values;
- **its rule figure** (section 7.8). A subsection with nothing judged gets a grey note saying why instead, and one no linkage group ran says so.

The cover's *At a glance* callout counts the groups each flag was raised on, the rule checks' among them: *2 of 12 linkage groups failed (DFM 1 group 3; DFM 2 group 1): wrong activation length in 2 groups, lockout broken in 1 group*, with the warnings counted the same way. When every group is clean on a firmware 4.0.0 recording it reads *Every linkage group's light was explained by its licks and kept its Paradigm's rule*.

7.8 Reading the rule figures

Each figure has one panel per DFM. Light that keeps the rule is grey-blue; light that breaks it is **red**, as bars and as a rug of ticks under the axis, so one failing Activation among hundreds still shows. The dashed lines are the rule's settings. Red means beyond the tolerance, as the checks judge.

- **Closed loop: lit time.** A histogram of the judged Activations' lit time, dashed at the Hold and, with it, at `MaxOnMs`. Healthy light sits at and to the right of the Hold (contact plus the Hold) and never past `MaxOnMs`; with `MaxOnMs` a pile of Activations sits on its line, the cut-offs. Red to the left of the Hold is *short activations*, red past `MaxOnMs` is *max on exceeded*.

- **Fixed length, interval: time between onsets, and lit time.** On the left, a histogram of the time from each Main onset to the one before, dashed at the longer of `LockoutMs` and `ActivationMs`; gaps more than four times that go in the last bar. Nothing should fall left of the line: red there is *lockout broken*. On the right, the lit time, dashed at `ActivationMs`: every Activation in the bar on the line. Red elsewhere is *wrong activation length*.
- **Fixed length, ratio: Requirement met, and lit time.** On the left, the Requirement each Main Activation met against its number K, one point per Activation spread a little across K, over the dashed series. Every point should sit on the line; large red points are *requirement series wrong*. A run resolved as starting one Activation early is drawn at K − 1. On the right, the lit time against `ActivationMs`, as for the interval branch.
- **Non-feeding: relight delay, and lit time.** A histogram of each Activation's delay after pyflic's last activity sample, dashed at `OnDelayMs`. It has no red, because no single delay fails: the median is `RelightDelayMs`. The bulk should sit within about a record of the line; a bulk well away from it is an `OnDelayMs` other than `Program.txt`'s. With `MaxOnMs`, the lit time beside it is dashed there, and red past it is *max on exceeded*.
- **Open loop** has no figure: the Program timeline shows its schedule (section 6.8).

---

8. Progressive Ratio experiments

A Progressive Ratio experiment gets everything above, including the optogenetic light QC and, on firmware 4.0.0, its rule checks, and a QC of its own, which asks whether the paired fly earned each light event. It is described in full in the Progressive Ratio guide, sections 5.1 to 5.3 and 6. A chamber group is a pair of chambers, one paired and one yoked, or in a design without yoked controls a single chamber (a solo group); the guide says where the groups come from. In brief:

Check	Verdict
Training never completed	removes the group's chambers (`require_training_complete`)
No training information: recorded by PyMCU 4.0.0 from a DFM whose firmware is older than 4.0.0	nothing removed: each group is one `Recording` facet, with no Test phase and no breaking point
Self-triggered light: five or more Test light events in a row with no sucrose lick since the previous one	**fails** the group
Implausible training: training completed, with light events, but without one sucrose lick	**fails** the group
Requirement series wrong (firmware 4.0.0): a Test light event met a Requirement the Test Paradigm's series does not give	**fails** the group
No increasing trend in licks per light event (not judged under a fixed ratio, whose requirement never rises)	warning

Check	Verdict
Resting level rise or **resting level elevated** at the sucrose well	warning

A group that fails leaves the analysis by default (`exclude_failed_pr_groups: true`): both chambers of a pair, the one chamber of a solo group. A light event is lick-free only when it has no sucrose lick since the previous one and no lick or touch within its Hold, so a light the fly touched for is never counted against it. On a firmware 4.0.0 recording a light event is the paired sucrose well's own Activation, and Training is read from its Phase rather than from a flag in the signal.

Requirement series wrong is the optogenetic light QC's own check (section 7.4), run on the Main Activations of the paired sucrose well inside the group's Test window, against the Test Paradigm `Program.txt` defines. It needs a firmware 4.0.0 recording and a ratio Test Paradigm in the program. The reason says what it means: *Program.txt is probably not the program that ran*.

A DFM restart inside a group's Test window ends that window (firmware 4.0.0, section 7.6). The ratio starts again after a restart, so the breaking point is judged up to the restart, and censored when the fly was still responding there. The group's `Notes` in `pr_light_qc.csv`, `summary.txt` and the report say so: *the DFM restarted at minute 412.3, 380.1 min after training end: the Test window ends at the restart, ….*

Output	What it is
`analysis/pr_light_qc.csv`	one row per chamber group: flags, verdict, `Excluded`, `LickFreeRunStartMin` and `Notes`; `YokedChamber` is `NA` for a solo group
`analysis/pr_light_events.csv`	every Test light event, with its licks and whether it counts; on firmware 4.0.0 also the Requirement it met and the firmware's feeding bouts behind it
`analysis/pr_light_events_dfm<n>.png`	**Licks per light event**: a working progressive ratio climbs
`analysis/pr_resting_level_dfm<n>.png`	**Sucrose Well resting level**: a well that creeps up is a sensor problem
`analysis/pr_cumulative_licks_dfm<n>.png`	**Training-aligned traces**: light that keeps firing while the paired trace is flat is self-triggering

The Hub lists them in the QC panel's **Progressive Ratio** group, below the optogenetic ones, for any member whose Experiment Type includes the Progressive Ratio module: *Light QC table* (the two tables above, with every flagged group in the log), *Licks per light event* and *Sucrose Well resting level*. **Run QC** writes the ticked ones into `analysis/`. The training-aligned traces are a figure of the Plots panel's Progressive Ratio group.

The two light QCs can disagree, and both are right. The PR check counts light *events*; the optogenetic check measures lit *time*. On the first real dataset, one group lit for four hours during training passed the PR check, because one long light event is not a run of lick-free events. It failed the optogenetic check, because 93% of those four hours had no lick behind them. Read both before deciding.

---

9. Projects

A Project pools its members, and so it pools their QC. **Combine**, on the Project panel's Analysis card, writes the Combined Analysis into the Project's `analysis/` folder, among it:

File	Holds
`<project>_Excluded.csv`	every member's exclusions: `manual` (its `remove_chambers.csv`) and `auto` (its `removed_chambers.csv`), with the reason
`<project>_LightQC.csv`	every member's light QC tables stacked, with a `Source` column: `PR` for a Progressive Ratio chamber group, `Opto` for an optogenetic linkage group, whose rows carry the rule checks' columns (section 7.7)
`<project>_Stats.txt`	the pooled statistics, with every flagged group listed before the tables

Every flagged group is listed in the Stats text and the Project Report's Quality control section with a **status**:

- **excluded**: its member's auto-removal took its chambers out;
- **retained**: it failed, but its exclusion switch was off, so it **is** in the pooled numbers;
- **kept — warning only**: it was only warned.

Pooling needs **every member judged by the same rules**. A Project's design owns the settings that decide QC: the detection parameters, the exclusion cutoffs and the light QC thresholds. Every member inherits them. Set them there, once, and keep a DFM's own settings to what its rig requires, such as `pi_direction`.

Before pooling, compare the exclusion counts per treatment in `<project>_Excluded.csv`. If one treatment lost far more chambers than another, the chambers that remain may not be comparable. Report the counts.

The Project Report's member table names the **generation** of each member's recordings, and adds a note for a member with no training information, so a Project that mixes firmware versions says so. A firmware 4.0.0 member contributes light measures an older one cannot (section 6.2).

---

10. A QC checklist

For each recording, before you look at any result:

1. **Load it and write the QC bundle**: Hub → QC → **Run QC**, every box ticked.
2. **Integrity**: every DFM *ok*, or every warning explained, lost records among them (section 3.1). Note each DFM's generation.
3. **Data breaks**: none long enough to matter (section 3.2).
4. **Raw signal**: no dead wells you did not expect, no wells drifting far (section 3.3).
5. **Baselined signal**: the thresholds sit sensibly for your data.
6. **QC Viewer**: detected bouts match the trace in a few chambers you can read by eye. Settle any parameter change now, and write it into the configuration (section 4).
7. **Two-well**: the bleeding check is *ok*, the crosstalk before the correction is small, and you know whether the dual-feeding correction was on (sections 3.4 and 3.5).
8. **Optogenetic**: `Program.txt` is in `data/` — for a firmware 4.0.0 recording, no *without Program.txt* warning anywhere; every linkage group is ok, or every flagged group has a decision (section 6).

9. **Optogenetic, firmware 4.0.0**: the Program timeline shows the program you meant to run, with no restart band you cannot account for; every trigger well left Acclimation, and every yoked well's light matches its partner's (section 6.8).
10. **Optogenetic, firmware 4.0.0, the rule checks**: every *Rule check* subsection of the report is ok, or every flag has an explanation. A failure is first a question about `Program.txt`; settle it before anything else (section 7).
11. **Progressive Ratio**: the training table and the PR light QC (section 8).
12. **Exclusions**: declare them, click **Run analysis** again, and check `removed_chambers.csv` and the report's *Excluded chambers* table (section 5).
13. **Write the report** (**Run analysis** with *Experiment report (PDF)* ticked, or `pyflic report`) and read its Quality control section and cover before its results.

In a Project, finish with **Combine** and read the flagged groups and the exclusion counts per treatment (section 9).

---

11. Quick troubleshooting

You see	Likely cause	What to do
The QC Viewer's DFM tabs show no integrity files or plots	the QC bundle has not been written, or their boxes on the Hub's QC panel are unticked	**Run QC** in the viewer, or tick the boxes and **Run QC** on the Hub's QC panel
The baselined plot shows thresholds you changed	the QC bundle predates the change, or *Baselined signal plots* is unticked, so Run QC left the old plot	tick it, load the member again and **Run QC**
A QC check you expected has no box on the QC panel	the member cannot have it: a two-well check on a single-well layout, a light check without light, a Progressive Ratio check under a type without that module	read the line under the groups, and hover over it for the reasons
A box on the QC panel is greyed	the data cannot answer it: *Program timeline* with no `Program.txt` section and no firmware 4.0.0 data	copy the MCU's `Program.txt` into `data/` if the recording had one, and load the member again
A box unticked on one member is unticked on every member of the Project	a member's ticks are the Project's, kept in `project.yaml`	nothing is wrong: members are checked alike. Tick it again to bring it back for all
An integrity **warning**	data breaks, error flags, lost records or a skipping index	read the integrity text for which; look at the signal plots around that time
Lost records in the integrity report	the DFM lost records while the MCU was not reading it; `Messages.csv` says *Lost N records* (older PyMCU: *Skipped index*)	a short gap changes little; treat results across a long one with care, as for a data break

You see	Likely cause	What to do
A gap listed under *DFM restarts*	the DFM restarted on its own mid-recording (section 3.1)	check the DFM's power and cable; read *DFM restarted* below
No Ack with nothing lost or repeated	an earlier PyMCU build's acknowledgement, which the firmware counted as missed	nothing: no data were lost
Repeated records dropped at load	PyMCU wrote a block twice after a failed file write	nothing: the first copy is kept
A well with no deflections at all on the raw plot	an empty position or a dead well	exclude the chamber by hand if a fly was in it
The simultaneous-feeding table's `Both` is zero everywhere	the dual-feeding correction is on, and `Both` is counted after it	read `Both_pre`, the count before it (section 3.4)
A **bleeding** warning	one well's signal appears in another above `feeding_threshold`	look at the responding well's baselined trace while the signalling well is fed from
Many chambers removed for too few licks	the flies fed little, or `min_untransformed_licks_cutoff` is high for this assay	check the cumulative licks; set the cutoff in the design, for every member
The Hub says **re-run needed**	exclusions changed after the results were written	run the member's analysis again (**Run analysis**, or the Project's **Analyze all**)
A linkage group failed for **unexplained light**	most of its lit time had no lick or touch behind it	open *Light explained by licks*: orange beside the red is a drifting well; no orange is a light the firmware had no reason to switch on
Unexplained light only **warns**, never fails	no `Program.txt` in `data/`	copy the MCU's exported `Program.txt` into `data/` and run again
"holds more than one Program.txt"	two program files in `data/`	keep only the one exported for this recording
"does not satisfy experiment_type …: the Opto Program runs ClosedLoop" (or another Paradigm type)	the program runs light the Experiment Type does not expect (section 6.2)	check that `Program.txt` is this recording's; then choose a type that expects that light, or build one in the Experiment Type Editor

You see	Likely cause	What to do
No light recorded fails a DFM	`optogenetics: true`, and the DFM's light was never on	check the lid and the program; set the DFM's own `optogenetics: false` if it ran without a lid
Program mismatch	the data's light settings or start time differ from the program, or a row names another Paradigm than the program's line	check that `Program.txt` is the one exported for this recording
firmware 4.0.0 recording without Program.txt	the program was not copied into `data/`	copy the `Program.txt` PyMCU exported for this recording and run again
Paradigm not in program fails every group of a DFM	the rows name a Paradigm `Program.txt` does not define: the file is another recording's	find the `Program.txt` exported with this recording
Broken yoking fails a group	linked wells' light differs, or an `Off` well counted Activations: the yoked fly did not get its partner's light	check the lid's wiring and the program's `Linkage:` line; the Program timeline shows which well
Acclimation count mismatch	a trigger well's Acclimation counted other than its Paradigm's number	check that `Program.txt` is this recording's; then look at the well's lane in the Program timeline
Firmware events disagree	the firmware and pyflic counted feeding bouts differently at a trigger well	look at the baselined trace; compare the Paradigm's `EventGapMs` with `feeding_event_link_gap`
Unknown-instruction rows or **late records**	the DFM ran something outside the program, or records came late; a restart's rows under no instruction count among the first	note the minutes; those rows are not judged
Short activations, **max on exceeded**, **wrong activation length**, **lockout broken** or **requirement series wrong** fails a group	the light broke its Paradigm's rule, which the firmware keeps exactly: the `Program.txt` in `data/` is probably not the program that ran, or the LED or its `OnMs` report is faulty	find the `Program.txt` exported with this recording and run again; the reason names the setting, and the rule figure shows which Activations break it (sections 7.3, 7.4 and 7.8)
Contact without activation	pyflic saw licks or touches at a free FixedInterval well that brought no Activation: contact below the firmware's threshold, or a dead sensor or LED	look at the trigger well's baselined trace around the misses and at its threshold in `Program.txt`; a few are normal (section 7.4)

You see	Likely cause	What to do
Non-feeding light during feeding	a NonFeeding light stayed on while the fly fed	check the linkage and the lid; look at the trigger well's trace (section 7.5)
Non-feeding dark	a NonFeeding light stayed off where its rule says lit: a resting signal that drifted up, which the firmware reads as contact, or a longer `OnDelayMs` than `Program.txt` states	look at the trigger wells' strips in the Program timeline and their raw signal for drift (section 7.5)
`RelightDelayMs` far from `OnDelayMs`	the program that ran had another `OnDelayMs`; no flag watches it	check that `Program.txt` is this recording's (section 7.5)
Interval started late	the DFM took up a Program Interval's instruction late, and PyMCU sent it again	note the minutes; many late Intervals point at the DFM's connection (section 7.6)
DFM restarted	the DFM restarted mid-recording, after a power dip or a loose cable; its Paradigm started over	check its power and cable; results across the restart are two runs of the program, and a Progressive Ratio Test ends at it (sections 7.6 and 8)
Every rule check column is `NA`	no `Program.txt` section for the DFM, or a recording older than firmware 4.0.0	copy the `Program.txt` exported for this recording into `data/`; an older recording keeps the checks of section 6
A rule check subsection says nothing could be judged	every Activation was merged with other light or cut short, as held FixedInterval contact with a lockout shorter than `ActivationMs` does	it is not a failure, only light whose length cannot be read; the not-judged count is in the group's notes (section 7.1)
no training information on a Progressive Ratio member	recorded by PyMCU 4.0.0 from a DFM with older firmware	nothing to fix here: each group is one `Recording` facet; update the DFM for the next recording